

Discrete variable 3D models in Computer extended Descriptive Geometry (CeDG): Building of polygonal sheet-metal elbows and comparison against CAD

Manuel Prado-Velasco^{ID*}, Laura García-Ruesgas^{ID}

Department of Engineering Graphics, Engineering Faculty, University of Seville, ETSI, camino de los descubrimientos s/n, Seville, 41020, Spain
Multilevel Modelling and Emerging Technologies in Bioengineering, University of Seville, Spain

ARTICLE INFO

Keywords:

Computer extended Descriptive Geometry
Computer aided design
3D discrete modeling
Sheet metal
Polygonal elbows
Dynamic geometry software

ABSTRACT

The Computer extended Descriptive Geometry (CeDG) is as a novel approach based on Descriptive Geometry to build 3D models within the framework provided by Dynamic Geometry Software tools. Parametric CeDG models can be interactively explored when continuous parameters change, but this is not the case for discrete parameters. This study demonstrates the capability of the GeoGebra - CeDG approach to incorporate algorithms that build discrete variable 3D models with dynamic parameterization. Several 3D models and their flattened patterns (neutral fiber), based on a new developed CeDG algorithm, were compared to their LogiTRACE v.14 and Solid Edge 2024 (CAD) counterparts. The accuracy of the CeDG models surpassed that of CAD models for nearly all dimensions defined as metrics. In addition, the CeDG approach was the unique that provided an automatic solution for any value of the number of ferrules.

1. Introduction

The ascendancy and widespread adoption of Computer-Aided Design (CAD) in recent decades have progressively marginalized Descriptive Geometry (DG) within both academic curricula and professional practice [1]. This shift has prompted efforts to combine traditional DG techniques with CAD tools to enhance accuracy and achieve goals that would be difficult, if not impossible, without digital support [2,3].

A relatively new concept, *Augmented Graphic Thinking* (AGT), refers to the expansion of DG through the use of digital procedures and simulations [4]. For instance, the drawing of geodesic lines on a non-developable surface by strips of rectifying surfaces is cited as an example of a complex operation that was deduced using AGT [5].

The impact of computers on the methodologies and applications of DG has been the subject of considerable research, with studies highlighting the potential for harnessing the synergies between image synthesis and solution analysis, as well as advancing into the construction processes of geometric loci [1]. However, to the best of our knowledge, there is currently no novel computer-based methodology that fully accounts for this synergy. Another area that has experienced significant advancements in recent decades is dynamic geometry, particularly in the mathematical domain [6]. Recently, there has been a growing interest in integrating dynamic geometry into university-level

education, as evidenced by the emergence of new teaching approaches in this field [7]. It is noteworthy that the initial exploration of the use of DGS to support DG can be traced back to the thesis work of Wottreng [8], in which the influence of Gaspard Monge's developments on descriptive and differential geometry was analyzed. Wottreng employed one of the inaugural dynamic geometry software (DGS) programs, Sketchpad, to facilitate the movement of an auxiliary cutting plane during the intersection of two cones, thereby generating a computational locus. Sketchpad, which was developed by Sutherland as part of his doctoral dissertation [9], is regarded as a foundational precursor to the existing paradigm of CAD.

The concept of *Computer-extended Descriptive Geometry* (CeDG) emerges as a computer-based methodology rooted in DG for 3D modeling. It is designed to serve both academic and professional communities and is aligned with the principles of AGT. However, CeDG requires implementation within a Dynamic Geometry Software (DGS) tool. Developed in the early 2020s, CeDG responds to the displacement of DG by modern CAD systems. It leverages the natural relationship between the sequence of mathematical entities in a DGS model and the sequence of graphical procedures in DG [10]. As a result, CeDG models integrate graphical and projective objects in their fundamental mathematical forms, in contrast to CAD, which focuses on controlling

* Corresponding author at: Department of Engineering Graphics, Engineering Faculty, University of Seville, ETSI, camino de los descubrimientos s/n, Seville, 41020, Spain.

E-mail addresses: mpradov@us.es (M. Prado-Velasco), lauragr@us.es (L. García-Ruesgas).

<https://doi.org/10.1016/j.gmod.2024.101253>

Received 21 September 2024; Received in revised form 20 November 2024; Accepted 24 December 2024

1524-0703/© 2025 The Authors. Published by Elsevier Inc. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

3D shapes through B-Splines.

Unlike CAD, which uses surface-curve approximations [11], CeDG offers a fundamentally different approach. A CeDG model consists of a sequence of graphic-mathematical entities that are interrelated through specific dependencies. These entities represent the primitives and curves derived from the projections of a 3D system, along with the procedures of Descriptive Geometry and other mathematical relationships. This organizational structure allows for precise and efficient representation of complex 3D systems by integrating a variety of mathematical and geometric techniques.

CeDG models are implemented using Dynamic Geometry software (DGS) platforms, such as GeoGebra, which provides an interactive environment for the exploration and manipulation of these graphic-mathematical entities [6,12,13]. GeoGebra facilitates the creation of dynamic parametric models, enabling users to visualize and adjust projections and curves in real-time. This capability enhances the analysis of the dynamic behavior of geometric constructions while preserving their integrity. Furthermore, it allows designers and engineers to observe how changes in parameters impact the overall structure.

Nevertheless, a more intricate challenge arises when parameters are discrete and lead to qualitative, non-continuous changes in the 3D model. To address this challenge, a methodology for constructing 3D CeDG models that incorporate qualitative changes associated with discrete parameters must be developed. One practical example of this approach is the creation of polygonal elbows (both cylindrical and conical) made from n -ferrules, which can adapt to changes in the discrete variable n (where $n > 2$).

Thus, the goal of this study is twofold: (i) to demonstrate the capability of the CeDG approach in building 3D models with dynamic parameterization for discrete variables, through the development of a novel algorithm that supports the automatic generation of cylindrical and conical polygonal elbows and their flattened patterns, and (ii) to compare these models with those created using industry-standard software tools - LogiTRACE v.14 (specialized in Sheet Metal) and Solid Edge 2024 (a general-purpose software with a sheet metal module)- in terms of scope, construction methodology, and precision.

2. Methods and software

We use GeoGebra (<https://www.geogebra.org>), a dynamic geometry software, to implement the CeDG approach. The algorithms for generating discrete-variable polygonal elbows are implemented using the GeoGebraScript language, which is available in GeoGebra Classic 5 (desktop-oriented) and GeoGebra Classic 6 (web-oriented). GeoGebra can be run either through its executable file (GeoGebra.exe) or as a web service. Although GeoGebra's installers are free for non-commercial use, its source code is licensed under the GNU General Public License (v3). For broader usage, including commercial applications, GeoGebra can be executed through the Gradle build tool (<https://gradle.org/>) or any other tool that utilizes the GeoGebra source files directly. This approach offers more flexibility for commercial deployment and extension.

The reliability of the novel algorithms, parameterized with respect to continuous variables (dimensions) and discrete variables (number of ferrules), was analyzed using both a cylindrical elbow and a conical elbow. The reference nominal dimensions were as follows: diameter (a unique value for the cylindrical elbow and the first value for the conical elbow), $D_1 = 150$ mm, second diameter for the conical elbow, $D_2 = 70$ mm, elbow radius, $R_{elb} = 300$ mm, and elbow angle, $\delta = 98^\circ$. Sheet metal models were developed using the neutral surface, serving as a preliminary step in creating the graphical pattern of a sheet metal part. This methodology mitigates dependency on material type and emphasizes surface geometry in both 3D and flattened forms. The associated CeDG models include the primary orthographic view and its flattened state (flat pattern). Reliability was assessed by confirming that these models adapt dynamically to variations in the number of ferrules

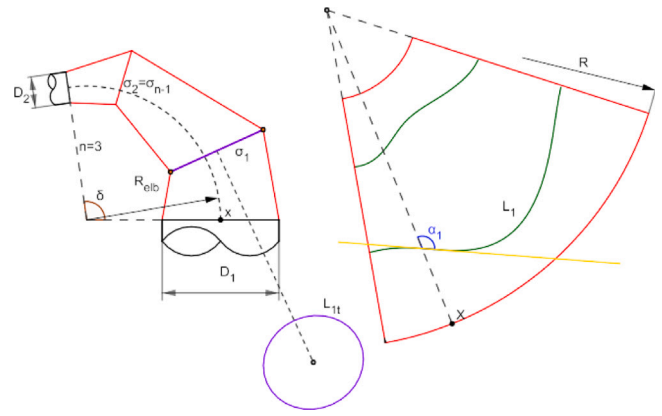


Fig. 1. Canonical projection of a conical polygonal elbow model (see text).

(greater than or equal to two) and continuous dimensions. Additionally, the models were constructed using Solid Edge 2024 and LogiTRACE v.14, to compare their reliability against the CeDG approach.

The accuracy of CeDG, Solid Edge 2024 and LogiTRACE v.14 models was evaluated by comparing the main dimensions of their flat patterns. These dimensions are illustrated in Fig. 1, using the example of a conical elbow with its mouths positioned perpendicular to the projection plane and centered within a plane parallel to this same projection plane (canonical projection). The key dimensions selected for this comparison included the following: the length of the flattened section curve between ferrules i and $i+1$, denoted as L_i , the radius R of the circular arch representing the cone of revolution from which the ferrules are derived, and the angle α_1 between the generatrix through X and the tangent line, which serves as a local index of the curvature at the first flattened section curve (the inflection point). The true values for these dimensions were calculated based on the system's geometric properties. The value of R was determined through descriptive geometry procedures [14], whereas the true value of L_i corresponds to the perimeter of the spatial plane section (illustrated in Fig. 1 for the first section, denoted as L_{1r}). The angle α_1 represents the angle between the 3D generatrix through X and the tangent to the plane section at X , leveraging its invariance in the flattening process.

For cylindrical elbows, the selected dimensions included the length of the flattened section curve between ferrules, denoted as L , and the angle α_1 . In cases where the model provides different values for L , the mean and standard deviation of these values will be reported.

The numerical comparison was conducted for elbows with 3, 5, and 7 ferrules. Accuracy was quantified using the Relative Error (RE), defined as $RE = (M-T)/T \cdot 100\%$, where M represents the model measurement and T is the true value. For cases with multiple measurements of the same true value, accuracy was assessed using the Root Mean Square Error (RMSE), calculated as $RMSE = \sqrt{1/n \sum_i (M_i/T - 1)^2} \cdot 100\%$, where M_i and T represent the measurement values and the true value, respectively. Sensitivity analyses were also conducted to ensure the robustness of the models.

3. Algorithms for the generation of polygonal elbows in CeDG

To create a model parameterized by both continuous (dimensions) and discrete (number of ferrules) variables for modeling of cylindrical elbows, we begin by analyzing two paradigms. In the *first paradigm*, the model is constructed using iterative and control flow structures, which must be updated if any parameter changes. In contrast, the *second paradigm* involves building only dynamic objects at each coding stage. These objects maintain interdependence based on the construction sequence and automatically respond to parameter changes without requiring an explicit re-execution.

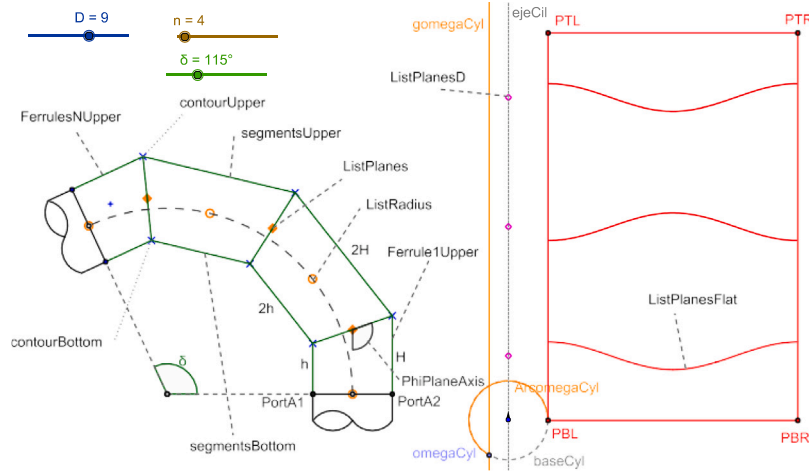


Fig. 2. Modeling a discrete variable cylindrical polygonal elbow with algorithm-generated auxiliary objects.

3.1. First paradigm

We begin with the algorithm for constructing a cylindrical polygonal elbow. Fig. 2 presents a generic cylindrical elbow through its canonical projection (refer to the Methods section). This position simplifies the model construction, allowing it to be subsequently moved to any other projection view [10]. An additional orthogonal view is unnecessary in this canonical position. The model will also include the flattened state of the elbow (Fig. 2, right).

Considering that the ferrules constituting the elbow can be derived from a single revolution cylinder through 180° rotation between consecutive pieces [14], the following GeoGebraScript code computes the sequence of bottom and upper contour points as depicted in the canonical projection in Fig. 2.

```

1  # First definitions
2  PortA1 = If(Distance(ET_1,EC)<Distance(ET_2,EC),ET_1,
   ET_2)
3  PortA2 = If(PortA1==ET_1, ET_2, ET_1)
4  CenterArcElbow = EC
5  PortB1 = Rotate(PortA1,ElbowAngle, CenterArcElbow)
6  PortB2 = PortB1 + UnitVector(Vector(CenterArcElbow,
   PortB1)) Distance(PortA1,PortA2)
7  # Tangent points
8  PortACenter = Midpoint( PortA1, PortA2 )
9  PortBCenter = Midpoint( PortB1, PortB2 )
10 linePortA = Line(PortA1, PortA2)
11 linePortB = Line(PortB1, PortB2)
12 Ang2dElbow = DeltaElbowAB / (nFerrules - 1)
13 ListRadius = IterationList(Rotate(ArcElbowPoint,
   Ang2dElbow, CenterArcElbow), ArcElbowPoint, {
   PortACenter}, nFerrules - 1)
14 ListPlanes = Sequence(Intersect(Tangent(ListRadius(
   iFerrules), ArcElbow), Tangent(ListRadius(
   iFerrules+1), ArcElbow)), iFerrules, 1, nFerrules
   - 1)
15 # Contour points
16 ListContourA1 = {}
17 SetValue(ListContourA1, 1, PortA1)
18 ListContourA2 = {}
19 SetValue(ListContourA2, 1, PortA2)
20 iFerrules = 1
21 linePlanes = Line(CenterArcElbow, ListPlanes(iFerrules
   ))
22 lineTangent = Tangent(ListRadius(iFerrules), ArcElbow)
23 DO = {"SetValue(ListContourA1, iFerrules+1, Intersect(
   linePlanes, Line(ListContourA1(iFerrules),
   lineTangent)))", "SetValue(ListContourA2,
   iFerrules+1, Intersect(linePlanes, Line(
   ListContourA2(iFerrules), lineTangent)))", "
   SetValue(iFerrules, iFerrules+1)"}
24 LOOP = {"If(iFerrules<nFerrules,Execute(Join(DO, LOOP)
   ))"}

```

```

25 Execute(LOOP)
26 SetValue(ListContourA1, nFerrules+1, PortB1)
27 SetValue(ListContourA2, nFerrules+1, PortB2)

```

GGB code 1 cylindrical elbow (first paradigm)

Lines 1 to 6 arrange the mouth ends to ensure that portA1 and PortB1 are positioned on the bottom contour, while PortA2 and PortB2 are on the upper contour. The following block divides the elbow arch - a circular arch between the mouth centers - into $n - 1$ segments, where n (referred to as $nFerrules$) represents the number of ferrules. The $ListRadius$ points are the intersections between these division lines and the arch (marked by empty circles in Fig. 2). Additionally, $ListPlanes$ denotes the sequence of points created by the intersection of tangents to the arch at each $ListRadius$ point.

Planes perpendicular to the projection plane define sections between adjacent ferrules, using lines that originate from the arch center and intersect with points in the $ListPlanes$ sequence. The final block constructs these sections using the $linePlanes$ and $lineTangent$ objects within a $LOOP$ iterative structure, defined by the DO string list. As shown line 24, $LOOP$ references itself, making $Execute(LOOP)$ a recursive instruction that terminates when $iFerrules$ equals $nFerrules$.

The model's canonical projection is completed by another recursive $LOOP$ that generates the final contour segments and section planes. These can be accessed in the $CylElbowProj1Paradigm$ file in the supplementary material. As shown in Fig. 2, all intermediate ferrules are twice the size of the ferrules connected to the mouths. The model parameters - D (diameter of mouths), δ , and n - are linked to adjustable sliders; however, the Execution block requires manual initiation whenever these parameters change, which limits the model's dynamic updating.

The algorithm for flattening the elbow, following the initial paradigm, is presented below.

```

1  # Initialization
2  ejeCil = PerpendicularLine(PAC, PatRef)
3  vEje = UnitVector(ejeCil)
4  vRef = -PerpendicularVector(vEje)
5  dEje = Distance(ListPlanes(1),ListPlanes(2))
6  # Rectangle
7  PBL = PAC + Distance(PortA1,PortA2)/2 vRef
8  PBR = PBL + pi Distance(PortA1, PortA2) vRef
9  PTL = PBL + (nFerrules-1) dEje vEje
10 PTR = PBR + (nFerrules-1) dEje vEje
11 Segment(PBL, PBR)
12 Segment(PTL, PTR)
13 Segment(PBL, PTL)
14 Segment(PBR, PTR)
15 # Planes' sections

```

```

16 ListPlanesD = IterationList(PEje + dEje vEje, PEje, {
    PAC + dEje/2 vEje}, nFerrules - 2)
17 PhiPlaneAxis = Angle(PortACenter, ListPlanes(1),
    ListContourA2(2))
18 # Flattened section curves
19 baseCyl = Circle(PAC, PBL)
20 omegaCyl = Point(baseCyl)
21 gomegaCyl = Line(omegaCyl, ejeCil)
22 ArcomegaCyl = Arc(baseCyl, PBL, omegaCyl)
23 iFerrules = 1
24 DO = {"Locus(Intersect(PerpendicularLine(Intersect(
    Line(ListPlanesD("+iFerrules+"), Rotate(PAC, (-1)
    ~ + iFerrules+"*PhiPlaneAxis, ListPlanesD("+
    iFerrules+"))), gomegaCyl), ejeCil), Line(PBL +
    Length(ArcomegaCyl) vRef, ejeCil)), omegaCyl)", "
    SetValue(iFerrules, iFerrules+1)"}
25 LOOP = {"If(iFerrules<=nFerrules-1, Execute(Join(DO,
    LOOP)))"}
26 Execute(LOOP)

```

GGB code 2 pattern of cylindrical elbow (first paradigm)

We begin by defining the cylinder's axis, $ejeCil$, based on the reference point PAC and perpendicular to $PatRef$ (line 2), as illustrated in Fig. 2 (right). The axis length of an intermediate ferrule is calculated as $dEje$ (line 5). These elements enable the calculation of the rectangle that represents the full pattern, defined by points PBL , PBR , PTL and PTR (block Rectangle). To compute the flattened ferrules, we determine the points where section planes intersect $ejeCil$, labeled as $ListPlanesD$, as well as the angle between the section planes and the cylinder axis, $PhiPlaneAxis$ (lines 16–17).

The reference point $omegaCyl$ is then established on the support curve $baseCyl$ (lines 19–20). This point facilitates the computation of the flattened transforms of each curve section, using a recursive LOOP to generate them as parametric locus curves, in accordance with Eq. (A.1) (see Appendix A). Here, P_r represents the intersection of the $gomegaCyl$ generatrix with each section plane, align with corresponding points in $ListPlanesD$ (line 24) and mapped to the flattened rectangle for each ferrule.

As illustrated in Fig. 2, the pattern for a four-ferrule cylindrical elbow requires three plane curves to define the four ferrules. As expected, the flattened section curves are symmetric due to the 180° rotation between consecutive ferrules [14]. However, any change to a parameter value requires manual execution of the algorithm for updates, limiting dynamic parameterization. The complete code is provided in the *CylElbowPatt1Paradigm* file in the supplementary material.

3.2. Second paradigm

To eliminate the need for manually executing the algorithm, all Execute commands must be removed. In the updated code, the contour points are now calculated using the Sequence(command, i, 1, n) object. This approach dynamically generates a list based on command(i) when the index i ranges from 1 to n . This internal command replicates the logic of GGB code 1 (lines 16–27), ensuring that the results update automatically without requiring explicit execution.

```

1 # Contour points
2 contourBottom = Sequence(Intersect(Line(ListRadius(
    iFerrules) + Distance(PortACenter, PortA1)
    UnitVector(Vector(ListRadius(iFerrules),
    CenterArcElbow)), Tangent(ListRadius(iFerrules),
    ArcElbow)), Line(CenterArcElbow, ListPlanes(
    iFerrules))), iFerrules, 1, nFerrules - 1)
3 contourUpper = Sequence(Intersect(Line(ListRadius(
    iFerrules) - Distance(PortACenter, PortA2)
    UnitVector(Vector(ListRadius(iFerrules),
    CenterArcElbow)), Tangent(ListRadius(iFerrules),
    ArcElbow)), Line(CenterArcElbow, ListPlanes(
    iFerrules))), iFerrules, 1, nFerrules - 1)

```

GGB code 3 cylindrical elbow (second paradigm)

A similar approach allows us to replace the recursive LOOP in GGB code 2 to construct the parametric locus curves, as demonstrated below.

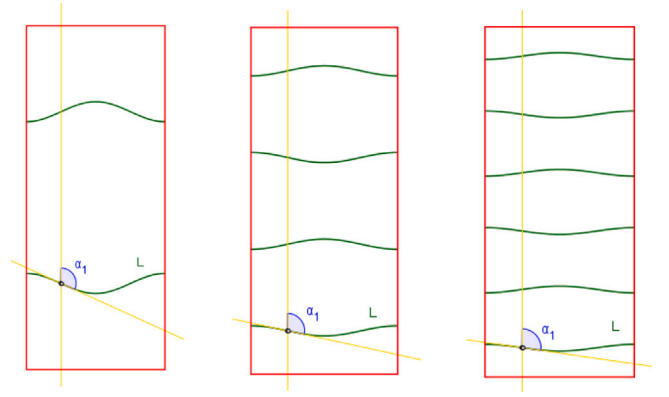


Fig. 3. Flat patterns of cylindrical elbows with 3-ferrule (left), 5-ferrule (center), and 7-ferrule (right) configurations, computed using the CeDG approach.

```

1 # Flattened section curves
2 ListPlanesFlat = Sequence(Locus(Intersect(
    PerpendicularLine(Intersect(Line(ListPlanesD(
    iFerrules), Rotate(PAC, (-1)^iFerrules*(~
    PhiPlaneAxis), ListPlanesD(iFerrules))),
    gomegaCyl), ejeCil), Line(PBL + Length(
    ArcomegaCyl) vRef, ejeCil)), omegaCyl), iFerrules
    , 1, nFerrules)

```

GGB code 4 pattern of cylindrical elbow (second paradigm)

Models based on this type of algorithm can dynamically respond to changes in both continuous and discrete variables, making them an ideal choice. The complete source code for the algorithm applied to the cylindrical elbow is available in the *CylElbowAlg* file within the supplementary material. A discrete variable model for a general polygonal conical elbow can also be constructed using the same algorithmic approach, as demonstrated in Appendix B.

4. CeDG models for cylindrical and conical elbows

The algorithms in the previous section were executed on GeoGebra Classic 5 to develop CeDG models for the cylindrical and conical polygonal elbows defined in the Methods section. Fig. 3 shows the flat patterns calculated for the three cylindrical elbows using the specified nominal dimensions. The values reached for α_1 and L (see Table 1) show that the models yield exact values for the defined metrics, consistent with the capability of the CeDG approach to maintain geometric integrity during the flattening transformation.

The conical elbow models parameterized using the nominal dimensions values, produced the flat patterns shown in Fig. 4. To facilitate inspection and comparison of the flattened geometry in the paper, the patterns are shown at different scales. Full-sized patterns are available in PDF files in the supplementary material.

Tables 2–4 present selected dimension values of the conical elbow patterns. As seen, except for R , the values of α_1 and L_i show minor differences from the true parameters. Although the relative errors are negligible, these differences are discussed below, as CeDG maintains the geometrical integrity of these curves.

5. CAD models for cylindrical and conical elbows

The construction of cylindrical and conical elbow models, as presented in the previous section, is detailed below using CAD technology. The software utilized includes Solid Edge 2024, a parametric CAD program for three-dimensional parts, and Logitrace v.14, specialized engineering software designed for developing flat layouts of ducts, components, and tools used in ventilation and heating systems.

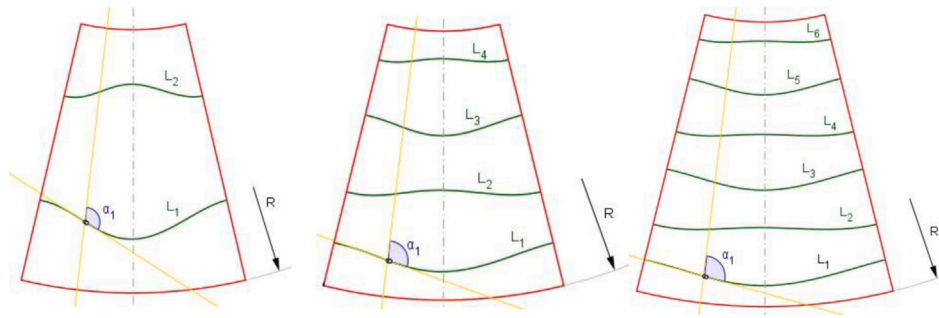


Fig. 4. Flat patterns of conical elbows with 3-ferrule (left), 5-ferrule (center), and 7-ferrule (right) configurations, computed using the CeDG approach.

Table 1

Lengths of sections between ferrules in flat patterns, L (mm)^a, together with α_1 (°) (see Fig. 1) and their relative errors^b (%)^c in cylindrical elbows.

Approach	3-ferrules		5-ferrules		7 ferrules	
	α_1	L	α_1	L	α_1	L
True	114.5	230.92	102.25	222.48	98.17	221.04
CeDG	114.5 (0)	230.92 (0)	102.25,(0)	222.48 (0)	98.17 (0)	221.04 (0)
LogiTRACE	114.5 (0)	230.67 ± 0.29 (0.15)	102.25 (0)	220.28 ± 0.16 (0.11)	98.24 (0.07)	220.93 ± 0.08 (0.06)
Solid Edge	114.32 (0.16)	230.93 (0)	102.17 (0.08)	222.49 (0)	98.17 (0)	221.05 (0)

^a In the case of several values for L , mean ± SD is written.

^b Root mean square error, RMSE (%), if there exist several measures for L .

^c Values < 0.01% are rounded to 0.

Table 2

Lengths of sections between ferrules in flat patterns, L_i (mm), together with α_1 (°) and R (mm) (see Fig. 1) and their relative errors (%)^a in 3-ferrules conical elbows.

Approach	α_1	R	L_1	L_2
True	114.6	1028.12	426.46	295.24
CeDG	116.13 (1.3)	1028.12 (0)	426.6 (0.03)	295.34 (0.03)
LogiTRACE	113.81 (0.69)	1028.12 (0)	426.42 (0)	295.2 (0.01)
Solid Edge	114.56 (0.03)	1026.37 (0.17)	425.16 (0.3)	294.33 (0.3)

^a Values < 0.01% are rounded to 0.

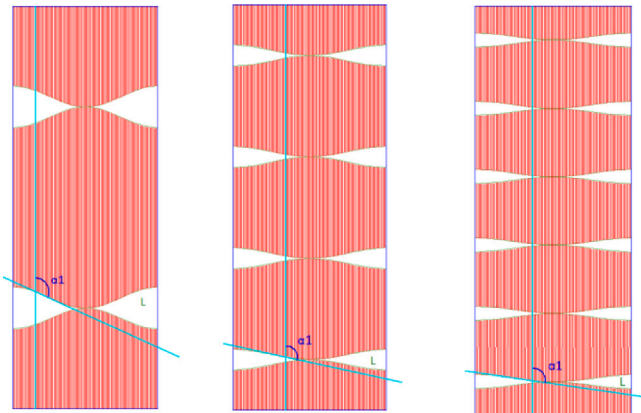


Fig. 5. Flat patterns of cylindrical elbows with 3-ferrule (left), 5-ferrule (center), and 7-ferrule (right) configurations, computed using the LogiTRACE approach.

5.1. LogiTRACE models

The construction of cylindrical elbows with 3, 5, and 7 ferrules was carried out in Logitrace using model number 066. Fig. 5 presents the flat patterns derived for these cylindrical elbows, based on the specified nominal dimensions.

Regarding the methodology, the program prompts the user to input essential design parameters, which in this case included the diameter of the elbow opening, the radius and angle of the elbow, the number of ferrules, thickness, and the number of generatrices. For this model,

a total of 120 generatrices and zero thickness were specified. No cylindrical supplements were added to adjust the pipe openings.

The model is displayed promptly in one of several 3D visualization formats, allowing for inspection of the intersections within the figures. This 3D visualization ensures that the shape conforms to the required specifications. Additionally, the shape is rendered in 2D, and a drawing with dimensions is generated and saved in the standard DXF format. The sheet's perimeter, weight, and area are calculated.

As illustrated in Table 1, the α_1 value for the 3- and 5-ferrule elbows is precise; however, for the 7-ferrule elbow, this value shows a slight deviation from the true value. For the L value, it was necessary to calculate the mean value in all cases, along with its root mean square error, as the ferrules did not all present the same value.

The construction of conical elbows with 3, 5, and 7 ferrules was performed using model number 120. Fig. 6 displays the resulting flat patterns. The data required by the application are identical to those for cylindrical elbows, with the only difference being that the diameters of the duct openings vary. A total of 120 generatrices and zero thickness were used. It should be noted that the flat patterns in Fig. 5 were computed without alternating the position of the joins, in contrast to those in Fig. 6.

As shown in Tables 2–4, the results obtained for L_i closely approximate the true values, while R values are exact. Scaled flat patterns are provided in DFT files within the supplementary material.

5.2. Solid edge models

The modeling of cylindrical elbows in Solid Edge requires using the “Sweep” tool within the “Surfaces” tab of the sheet metal module. This tool requires input data comprising a path and a cross-sectional

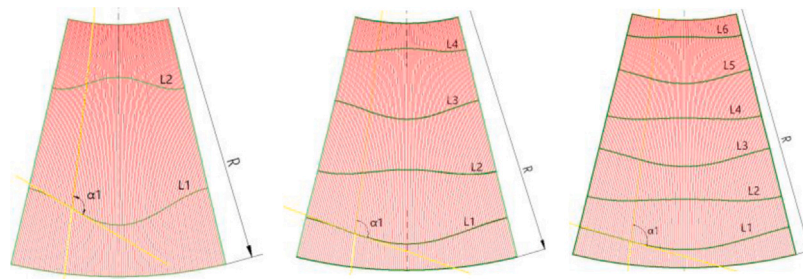


Fig. 6. Flat patterns of conical elbows with 3-ferrule (left), 5-ferrule (center), and 7-ferrule (right) configurations, computed using the LogiTRACE approach.

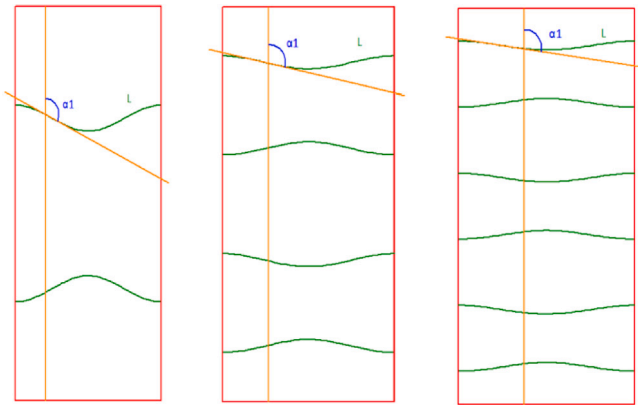


Fig. 7. Flat patterns of cylindrical elbows with 3-ferrule (left), 5-ferrule (center), and 7-ferrule (right) configurations, computed using the Solid Edge approach.

profile for the latter to follow. In this work, the cross-sectional profile is a circle with a parameterized diameter positioned in a plane perpendicular to the path. A nominal diameter of 70 mm was selected for the elbow opening, as specified in Section 2, with a small 0.01 mm opening added to allow for each cylindrical ferrule's flat layout pattern.

The path, drawn on the canonical projection (Fig. 1), is calculated based on the theoretical framework that determines the required number of ferrules from a single cylinder [15]. Since discrete variable parameterization is not possible in the projection, each elbow model was constructed manually.

To add thickness, each ferrule is individually created by following these steps:

1. Add a new sheet element named “ferrule₁” to the body.
2. Transfer the sheet element to the solid module to apply thickness, using the “Switch to” command in the “Tools” tab.
3. Within the “Sweep” command in the “Start” tab, add a minimum thickness of 0.01 mm, converting the first ferrule into a solid.
4. Repeat the process for each ferrule of the cylindrical elbow.

Once all segments are created, they are assembled using the “Multi-Body Publish” command. Each ferrule's file and the assembly file must be saved separately. To generate the flat pattern for the elbow ferrules:

1. Edit each ferrule to switch to the sheet module.
2. Select the “Thin Part to Sheet Metal” icon, converting it into a sheet part.
3. Choose the “Unfold” option, specifying the ferrule, edge, and origin to obtain the flat pattern.
4. Finally, generate a drawing to represent the flat pattern of the ferrule.

Using the defined nominal dimensions, flat patterns were calculated for cylindrical elbows with 3, 5, and 7 ferrules, as shown in Fig. 7.

Table 1 presents the resulting values for each case. The values of α_1 closely approximate the true values, with relative errors below 0.16%, while the L values match exactly.

In contrast, modeling the complete conical elbow within the Solid Edge sheet metal module proved unfeasible, as only a single ferrule can be generated using the “Loft” command. Therefore, each ferrule was individually generated in the solid module using the “Protrusion by Sections” command, which requires closed profiles. Profiles with a 0.01 mm thickness and a one-degree lateral opening were created and positioned on planes orthogonal to the canonical projection plane, defined by three points.

After generating each ferrule in the solids module, we proceed similarly to the cylindrical bends. By selecting the “Thin Part to Sheet Metal” icon, the thin sheet is converted into a sheet metal component. Using the “unfold” option, the flat pattern is obtained by selecting the ferrule, the edge, and origin for development. Following the modeling of conical elbows based on nominal dimension values, the flat patterns shown in Fig. 8 were derived. Scaled flat patterns are available as DFT files in the supplementary material. The values of α_1 , R, and L obtained from the flat patterns of the conical elbows are presented in Tables 2–4, showing slight deviations from the exact values.

6. Comparison of CAD and CeDG models

The first row of Table 1 presents the true values of the cylindrical elbow pattern dimensions, allowing for a comparison between the CeDG approach and traditional CAD. These values were obtained based on the geometric properties of the 3D model, as described in the Methods section.

CAD approaches yielded highly accurate values, with relative errors less than 0.16% in all cases. However, the CeDG approach proved even more accurate, with relative errors below 0.01% across all cases. In LogiTRACE, the length of the transformed section curves between adjacent ferrules (L) varied slightly depending on the ferrule. Table 1 displays the mean $\pm$ standard deviation for L values obtained using LogiTRACE to quantify this variation, with dispersions of 0.15, 0.11, and 0.06 mm for elbows with 3, 5 and 7 ferrules, respectively. In contrast, Solid Edge provided a single value for L, consistent across all ferrules of the same elbow and close to the true value.

The first rows of Tables 2–4 display the true values of selected dimensions for comparing conical elbow patterns, according to their geometric properties.

LogiTRACE provided the most accurate patterns for the 3- and 5-ferrule elbows, with slightly lower relative errors than other approaches. CeDG's accuracy was nearly equivalent to LogiTRACE for non-angular dimensions, with relative errors below 0.03%. Solid Edge provided the best accuracy for the angular values, with errors of 0.03% and 0% for the 3- and 5- ferrule elbows.

For the 7-ferrule elbow, CeDG was the most accurate approach, as shown in Table 4, though LogiTRACE accuracy was similarly high. Non-angular dimensions were calculated with greater precision than angular dimension.

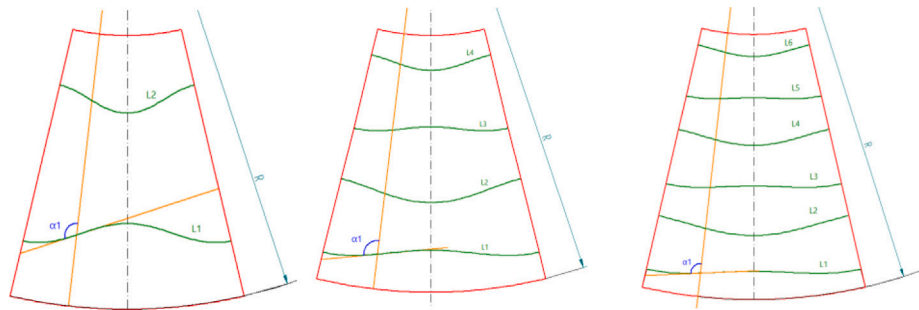


Fig. 8. Flat patterns of conical elbows with 3-ferrule (left), 5-ferrule (center), and 7-ferrule (right) configurations, computed using the Solid Edge approach.

Table 3
Lengths of sections between ferrules in flat patterns, L_i (mm), together with α_1 (°) and R (mm) (see Fig. 1) and their relative errors (%)^a in 5-ferrules conical elbows.

Approach	α_1	R	L_1	L_2	L_3	L_4
True	102.3	979.92	442.32	379.14	315.95	252.76
CeDG	103.16 (0.8)	979.92 (0)	442.37 (0)	379.17 (0)	315.98 (0)	252.78 (0)
LogiTRACE	102.24 (0.06)	979.92 (0)	442.28 (0)	379.09 (0.01)	316.05 (0.03)	252.72 (0.02)
Solid Edge	102.29 (0)	979.59 (0.03)	441.04 (0.3)	381.11 (0.5)	315.02 (0.3)	252.01 (0.3)

^a Values < 0.01% are rounded to 0.

Table 4
Lengths of sections between ferrules in flat patterns, L_i (mm), together with α_1 (°) and R (mm) (see Fig. 1) and their relative errors (%)^a in 7-ferrules conical elbows.

Approach	α_1	R	L_1	L_2	L_3	L_4	L_5	L_6
True	98.2	971.58	449.89	408.04	366.19	324.34	282.49	240.64
CeDG	98.75 (0.5)	971.58 (0)	449.91 (0)	408.06 (0)	366.21 (0)	324.36 (0)	282.5 (0)	240.65 (0)
LogiTRACE	98.82 (0.6)	971.58 (0)	449.84 (0.01)	408 (0)	366.14 (0.01)	324.3(0.01)	282.46 (0.01)	240.61 (0.01)
Solid Edge	98.9 (0.7)	972.25 (0.07)	448.62 (0.28)	406.88 (0.24)	365.16 (0.28)	323.42 (0.28)	281.65 (0.29)	239.92 (0.3)

^a Values < 0.01% are rounded to 0.

Due to the requirement to generate and unfold each ferrule individually in Solid Edge, the value of R was computed using the largest ferrule, as R is least sensitive to dimensional perturbations in this ferrule.

In summary, this study highlights three notable differences between CeDG and traditional CAD approaches:

1. Geometric Generation: CeDG algorithms for polygonal elbow generation are based on general descriptive geometry procedures and automatically generate the desired elbow and flattened pattern. In contrast, Solid Edge has limitations within its sheet metal module, requiring the manual drawing of the sketch of the main projection for each elbow model. LogiTRACE addresses this limitation by providing a set of sheet metal parts that can be combined into more complex systems, though it restricts polygonal elbows to a maximum of 12 ferrules.
2. Flattened Curve Integrity: CeDG maintains the integrity of the flattened curve patterns through the $L(\omega)$ functions from Eq. (A.1), unlike the B-spline-based curve approximations used in CAD. While the precision of B-splines is adequate for most engineering applications, maintaining geometric integrity allows for theoretical deductions that B-splines cannot support. The similar accuracy between CeDG and CAD, despite this difference, is discussed in the following section.
3. Dynamic Parameterization: CeDG supports dynamic parameterization of both continuous and discrete variables. Solid Edge only provides dynamic parameterization for continuous variables, as standard methods could not achieved a universal elbow. LogiTRACE does not support dynamic parameterization. A video demonstrating the dynamic parameterization of CeDG models of polygonal elbows is available in the supplementary material.

7. Discussion and conclusion

Numerous studies have shown that DG-based technical drawing offers a deeper understanding of spatial geometry compared to CAD [16, 17] and serves as a formal tool for solving spatial problems [4,18]. While the first assertion is well-established and supports the inclusion of DG in university programs, the second statement is more controversial. The proficiency of modern CAD tools in modeling 3D systems has contributed to the perception that DG is no longer relevant [19]. However, graphical procedures in DG align closely with a body of mathematical results related to the spatial properties of 3D systems. This alignment enables solutions to 3D geometry properties through deductive methods, in contrast to the trial-and-error approaches often required in traditional CAD tools.

Previous studies have demonstrated the reliability of CeDG in modeling 3D systems, showcasing new DG-based techniques for calculating the flat states of surfaces and their intersections, as well as solving physical problems related to graphical properties [10,20,21]. One major strength of CeDG against traditional CAD is its capability to use the corpus of DG knowledge as a formal language for 3D computer modeling, thereby overcoming the restriction of old manual technical drawing. CeDG leverages the mathematical foundation of geometrical DG procedures through the algebraic-graphical perspective of the DGS, enabling the formal deduction of geometrical solutions. A particularly relevant category of three-dimensional geometries are those expressed in terms of two-dimensional projections or transforms based on two-dimensional loci. This is exemplified by the flattened state of a curve, as given by Eq. (A.1). The field of loci computation is a significant topic in dynamic geometry, and recent theoretical developments are fueling the evolution of new generations of DGS [22]. One limitation of CeDG in comparison to CAD is the lack of solid-based operations like extrusion, sweep, or loft, which are integral to the primary workflow in traditional CAD. Additionally, the current state of CeDG is still evolving, suggesting that it may serve as a complementary tool to CAD.

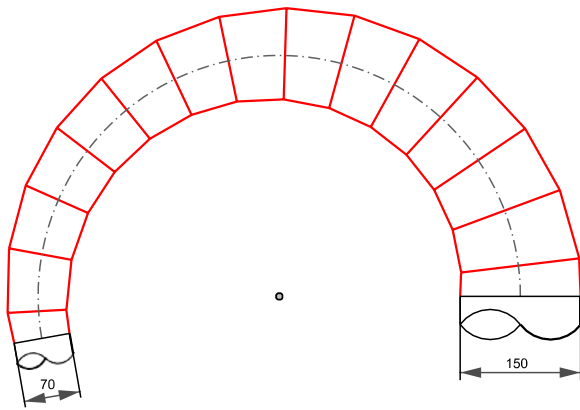


Fig. 9. Main projection (see Fig. 1) of a 15-ferrules elbow with $\delta = 190^\circ$ computed through CeDG.

In this study, we developed and implemented a DG-based technique in CeDG that generates discrete variable 3D models of cylindrical and conical polygonal sheet-metal elbows along with their flat patterns. The efficiency of these algorithms was evaluated through the creation of six elbows, which were also constructed using Solid Edge 2024 and LogiTRACE v.14 to provide a comparative analysis of the results.

Section 3 outlined the strategies that GeoGebraScript algorithms must fulfill to deliver automatic models that dynamically respond to changes in continuous and discrete variables. The algorithms were used to generate cylindrical and conical elbows, and we compared their flat patterns to those obtained using CAD approaches.

Our results demonstrate that both CeDG and CAD produce highly accurate patterns; however, CeDG uniquely avoids discrepancies in values for the same dimension (as seen with LogiTRACE for cylindrical elbows in Table 1 and Solid Edge for conical elbows where the largest ferrule dimension was selected for (R)).

Regarding automation, models in LogiTRACE and CeDG were constructed automatically based on the discrete and continuous parameter values. In contrast, Solid Edge faced challenges in building conical elbows under the sheet metal module, requiring a custom manual approach using the general part design module. Nevertheless, we computed the canonical projection of each elbow (Fig. 1- left) manually according to specific parameter values. This limitation is common in general CAD tools; however, automating model construction could be achieved by developing algorithms within the tool's supported programming languages and APIs. For instance, Rojas-Sola et al. [23] created a Visual Basic-based algorithm for CATIA that allows selection from pre-programmed polygonal elbows (ranging from 2 to 8 ferrules).

The superior performance of LogiTRACE over Solid Edge for sheet metal parts is expected, given LogiTRACE's specialization in that domain. However, LogiTRACE restricts the number of ferrules to 12 for designs exceeding 360 generatrices. While this limitation may be negligible for standard dimensions and tolerances, CeDG provides versatility by accommodating any number of ferrules. For example, Fig. 9 illustrates the main projection of a conical elbow with 15 ferrules for the specified angles and dimensions, with its flat pattern shown in Fig. 10.

The limitation of Solid Edge to automatically generate the elbow when the number of ferrules changes, the limitation of LogiTRACE to 12 ferrules and the fact that engineering designs rarely require more than 7 ferrules justify limiting the accuracy comparison to 7 ferrules. However, the behavior of LogiTRACE and CeDG up to 12 ferrules was similar to that presented here.

The dynamic response to changes in the number of ferrules and dimensions for any valid parameter value has been demonstrated and can be confirmed in the attached video. This feature allows to explore

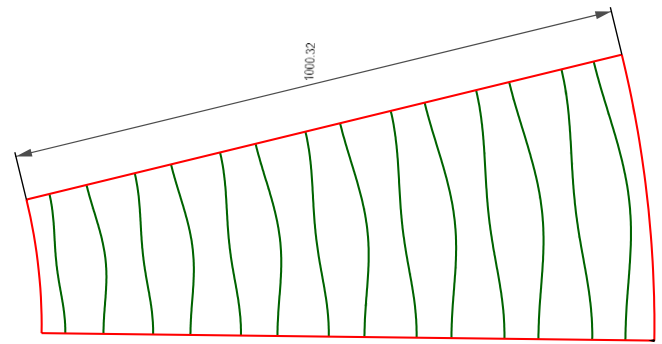


Fig. 10. CeDG flat pattern of the 15-ferrules elbow with $\delta = 190^\circ$ shown in Fig. 9.

the model system design space, the identification of a valid range for model parameters, and the definition of initial values for model variables associated with optimization tasks. For an illustrative study concerning the exploration of the design space, see [10]. The analysis of valid model parameters and optimization problems is described in [24, Chapters 2 and 8].

Despite the high degree of accuracy achieved by CeDG, the calculations for exact dimensions struggle to meet expectations, particularly for $L(\omega)$ (Eq. (A.1)). While relative errors for α_1 in conical elbows were approximately 1% (refer to Tables 2, 3, and 4), the relative errors for lengths (L_i) (length of the segments) were around 0.01%. The discrepancy can be attributed to the fact that $L(\omega)$ is derived from the Locus command, which generates a set of points whose distribution does not encompass the solution in a manner that is controlled by the user in DGS (GeoGebra). As α_1 is a local parameter, its accuracy is limited by the distances among these points. However, the values of L_i are integral, and their accuracy can be maintained as long as there are no significant changes in curvature and the number of points is sufficiently large. Since GeoGebra lacks a built-in command to compute the length of a Locus-based curve, a custom command was created using a recently developed GeoGebraScript-based tool [25]. This command processes the locus in two stages to improve length computation; however, the final accuracy remains constrained by the limitations of the Locus computation, explaining the non-zero relative errors observed in the 3-ferrules conical elbow.

We conclude that the new discrete variable algorithm accuracy models complies with the dynamic parametric behavior of CeDG models for constructing polygonal sheet-metal elbows and their patterns. These discrete variable conditional models represent a significant advancement in integrating discrete parameters within computer-extended descriptive geometry [26], expanding potential applications and increasing the utility of CeDG across various fields.

The selection of conical and cylindrical polygonal elbows as discrete variable 3D systems to model in CeDG was motivated by the fact that these systems are based on ruled single-curvature surfaces with numerous theoretical and practical applications in engineering [27]. Among these applications, the revolution polygonal elbows are particularly noteworthy. Moreover, the methodology underlying the algorithms developed in this study can be applied to other types of surfaces. Future research will focus on developing a specialized branch of Geogebra for CeDG that includes this type of algorithm and enhanced control over the accuracy of locus calculations and other dynamic objects.

CRediT authorship contribution statement

Manuel Prado-Velasco: Writing – review & editing, Writing – original draft, Validation, Software, Methodology, Conceptualization. **Laura García-Ruesgas:** Writing – review & editing, Writing – original draft, Software, Conceptualization.

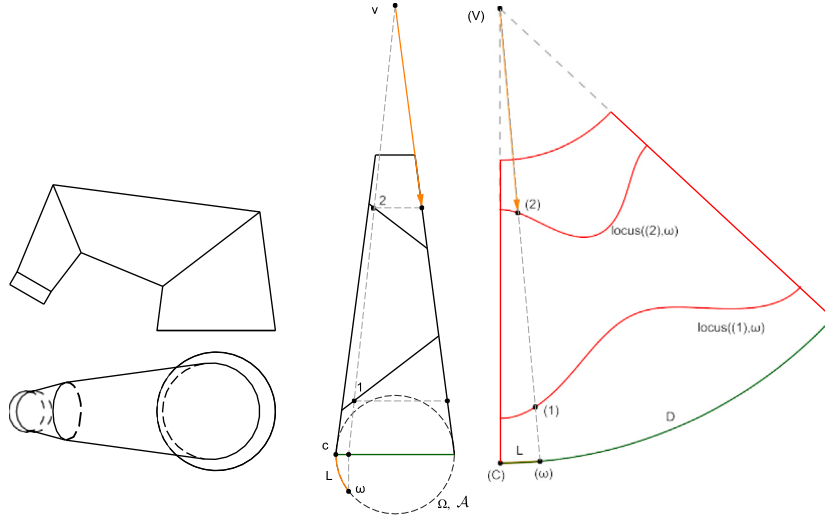


Fig. A.11. Orthographic views of a 3-ferrules conical elbow (left), revolution cone with plane sections from which the ferrules can be obtained (center), flattened transform of the revolution cone and section curves that separate the ferrules (right).

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Manuel Prado-Velasco reports financial support was provided by University of Seville. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported in part by the Universidad de Sevilla, Spain under Grants 2024/00000615 and 2024/00000544.

Appendix A. Cedg algorithm for surfaces' flattening

In CeDG, the flat pattern of any developable surface can be computed using the *locus* method. The equation for the flattened state, τ , of a curve C related to the surface is given by:

$$\tau \equiv L(\omega) = \text{locus}(P_\tau(f_D(\omega)), \omega \in \Omega), \quad (\text{A.1})$$

Here, $L(\omega)$ is a function that transforms points, ω , in the surface's plane section, Ω , to points, P_τ , of the flattened curve, while $f_D(\omega)$ is a mapping function matching points from Ω to the flattened transformed state, D , of the support curve $\mathcal{A}$. The curve $\mathcal{A}$ on the surface facilitates the transport of generatrices to its flattened state. For cylinders, $\mathcal{A}$ is defined as the intersection of the surface with a plane perpendicular to the axis, resulting in D being a straight line. A similar choice for $\mathcal{A}$ is used for revolution cones, where $\mathcal{A}$ corresponds to a circular arc for D . For any other conical surface, $\mathcal{A}$ can be defined as its intersection with a sphere centered at the cone's vertex, which results in D being a circular arc.

The function $L(\omega)$ is constructed using descriptive geometry procedures and, as such, it does not require to be algebraic. The flattened transform τ keeps the dynamic dependency on the parameters of C , according to the CeDG approach. A deeper analysis of Eq. (A.1) can be found in [21].

The application of Eq. (A.1) to flatten the revolution cone used in generating the 3-ferrules conical elbow shown in Fig. A.11 is presented below to clarify this technique.

Once a first generatrix of the developable surface is defined in the flattened domain (segment (V)-(C) in Fig. A.11 - right), the generatrix corresponding to ω is transported into the flattened domain. For revolution cones, $\mathcal{A}$ is defined as a plane section perpendicular to the axis,

making it equal to Ω , as illustrated.

Since the distance between c and ω along $\mathcal{A}$ corresponds to the distance between (C) and (ω) along D , the position of (ω) can be calculated, and the associated generatrix is then transported. Distances V-2 and V-1 (in 3D space) are preserved in the flattened state, enabling (2) and (1) to be positioned along the generatrix defined by (ω) . The section curves that divide the cone into three ferrules can then be flattened as *locus*((2), ω) and *locus*((1), ω) (Fig. A.11 - right), according to Eq. (A.1).

Appendix B. Algorithm for a discrete variable conical polygonal elbow model

We now consider the construction of a generic conical elbow with n ferrules and its flattened pattern, with parameters (see Section 2) linked to sliders to dynamically explore the behavior of the model. The diameter D_2 is controlled by the parameter *Dfrac* by means of the equation $D_2 = \text{Dfrac} \cdot D_1$. All ferrules comprising a conical polygonal elbow can also be obtained from a single cone by rotating adjacent pieces 180° [14]. The GGB code snippet below shows the main instructions for the algorithm that generates the orthogonal view in the canonical position.

```

1 # Points in elbow arc
2 Ang2dElbow = DeltaElbowAB / (nFerrules - 1)
3 ListRadius = IterationList(Rotate(ArcElbowPoint,
   Ang2dElbow, CenterArcElbow), ArcElbowPoint, {
   PortACenter}, nFerrules - 1)
4 ListSphereCenter = Sequence(Intersect(Tangent(
   ListRadius(iFerrules), ArcElbow), Tangent(
   ListRadius(iFerrules+1), ArcElbow)), iFerrules,
   1, nFerrules - 1)
5 # Spheres inscribed in cone
6 ejeCone = PerpendicularLine(PAC, PatRef)
7 vEje = UnitVector(ejeCone)
8 vBase = PerpendicularVector(vEje)
9 halfFerruleL = Distance(PortACenter, ListSphereCenter
   (1))
10 End1Gen = PAC + Distance(PortA1, PortA2)/2 vBase
11 End2Gen = PAC + 2 (nFerrules-1)halfFerruleL vEje +
   Distance(PortA1, PortA2)Dfrac/2 vBase
12 Gen = Line(End1Gen, End2Gen)
13 ListSphereCenterGen = IterationList(Pstart+2
   halfFerruleL vEje, Pstart, {PAC + halfFerruleL
   vEje}, nFerrules-2)
14 ListSphereTanGen = Sequence(Intersect(
   PerpendicularLine(ListSphereCenterGen(iFerrules),
   Gen), Gen), iFerrules, 1, nFerrules-1)
15 radSpheres = Sequence(Distance(ListSphereCenterGen(
   iFerrules), ListSphereTanGen(iFerrules)),
   iFerrules, 1, nFerrules-1)

```

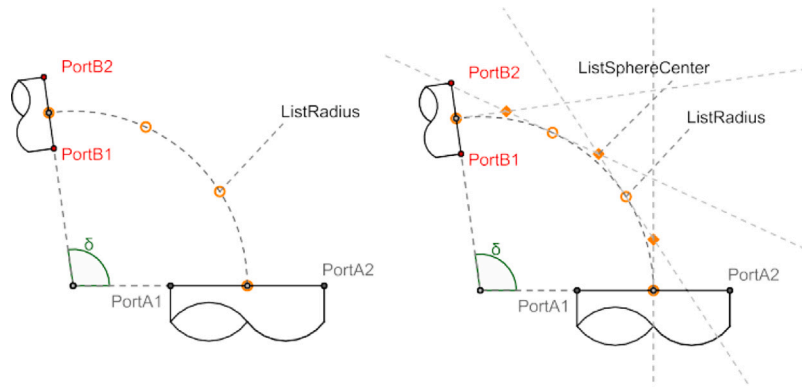


Fig. B.12. Discrete variable conical polygonal elbow model: division of the elbow arch (left) and centers of spheres inscribed in cone pieces (right).

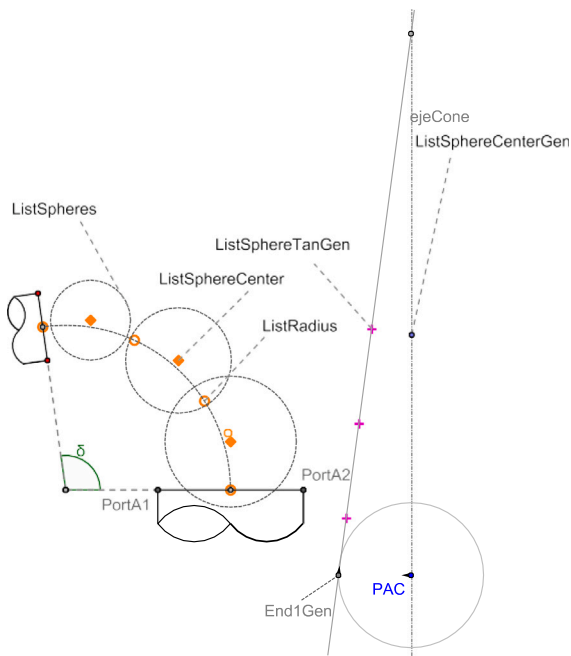


Fig. B.13. Discrete variable conical polygonal elbow model: determination of radii of spheres and their placement in the canonical view.

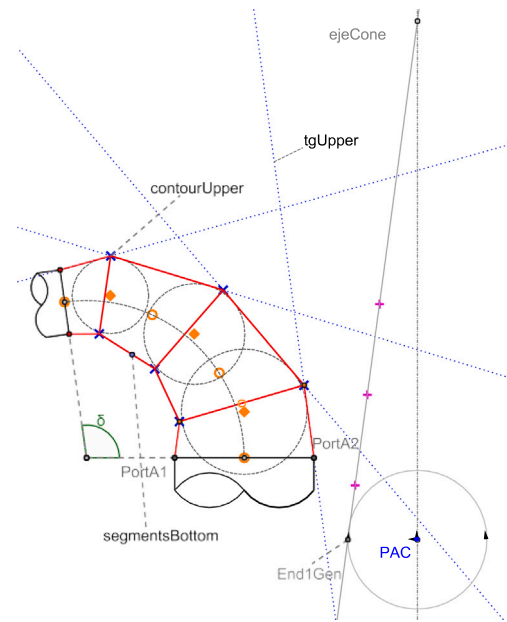


Fig. B.14. Discrete variable conical polygonal elbow model: external tangents between consecutive spheres that give the final ferrules in the canonical view.

```

16 ListSpheres = Sequence(Circle(ListSphereCenter(
    iFerrules), radSpheres(iFerrules)), iFerrules, 1,
    nFerrules-1)
17 # List of tangents to spheres and contour points
18 tgSpher = Sequence({Tangent(ListSpheres(iFerrules),
    ListSpheres(iFerrules+1))}, iFerrules, 1,
    nFerrules-2)
19 tgUpper = Join({Element({Tangent(PortA2, ListSpheres(1)
    }, 1)}, Sequence(Element(RemoveUndefined(Sequence
    (If(Element(Sequence(Intersect(Element(tgSpher(
    iFerrules), i), Segment(ListSphereCenter(
    iFerrules), ListSphereCenter(iFerrules+1))), i,
    1, Length(tgSpher(iFerrules))), j) <> Element(
    Sequence(Intersect(Element(tgSpher(iFerrules), i),
    Segment(ListSphereCenter(iFerrules),
    ListSphereCenter(iFerrules+1))), i, 1, Length(
    tgSpher(iFerrules))), j), Element(tgSpher(
    iFerrules), j)), j, 1, Length(tgSpher(iFerrules))
    ), 2), iFerrules, 1, nFerrules-2),
20 {Element({Tangent(PortB2, ListSpheres(nFerrules-1))}, 2)
    })
21 contourUpper = Sequence(Intersect(tgUpper(iFerrules),
    tgUpper(iFerrules+1)), iFerrules, 1, nFerrules)

```

GGB code 5 conical elbow (second paradigm)

The algorithm leverages the system's projective properties [14]. The first block of the code 5 (lines 2–4) generates the list of points dividing the elbow arch into $n - 1$ segments (where n is the number of ferrules), denoted as `ListRadius` (Fig. B.12 - left). These points are used to compute the centers of the spheres inscribed in the cone pieces, `ListSphereCenter`, by intersecting the tangents to the arch. (Fig. B.12 - right).

The radii of the inscribed spheres are determined through the construction of the cone that will be sectioned to form the ferrules.

This step is executed in the second block (lines 5–16). The algorithm constructs the cone axis, `ejeCone`, using the reference point, `PAC`, and a line perpendicular to the reference line `PatRef`, both of which are input parameters. The radii are calculated by intersecting the perpendiculars to the cone's left generatrix (`Gen`, the line defined by `End1Gen` and `End2Gen`) with lines passing through the sphere centers along the `ejeCone` (`ListSphereCenterGen`). This yields a list of points on the generatrix, `ListSphereTanGen`, such that the distances between these points and `ListSphereCenterGen` correspond to the sought radii, `radSpheres`. All referred objects are labeled in Fig. B.13 - right. Once the list of spheres, `ListSpheres`, is determined, they can be positioned in the canonical view of the conical elbow, as shown Fig. B.13.

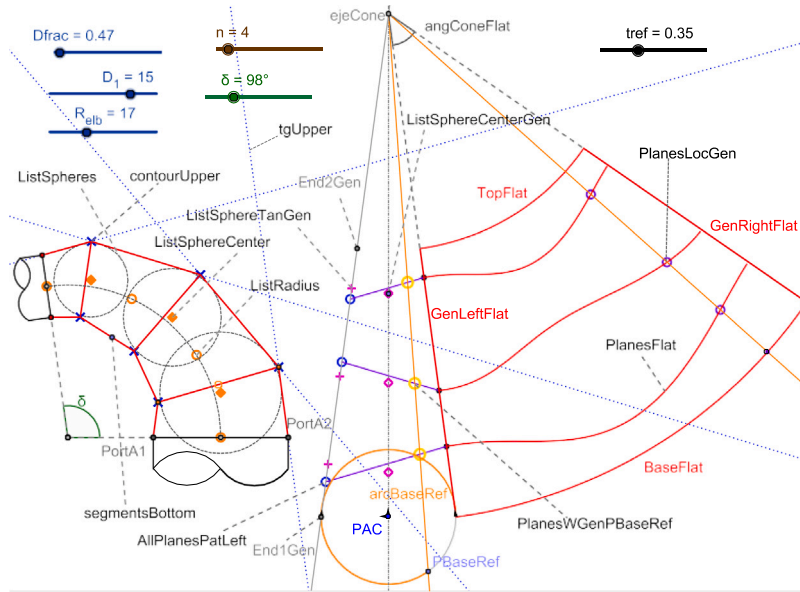


Fig. B.15. Discrete variable conical polygonal elbow model and its flattened pattern with the auxiliary objects built during the algorithmic process.

The last block of the algorithm (lines 17–21) completes the construction of the elbow by computing the external tangents between consecutive spheres (including the mouth ends for the first and last spheres). Since the classification of the four tangents between two spheres, computed using the *Tangent* command, is not predefined, the algorithm includes a logical structure to identify and exclude the internal tangents (those intersecting the line segment between the centers of the spheres). This is achieved using the *RemoveUndefined* command (line 19). The remaining tangents forming the upper contour are stored in the *tgUpper* list, and their intersections define the list of contour points along the upper generatrices, *contourUpper*. Lists *tgUpper* and *contourUpper* are labeled in Fig. B.14.

The bottom contour is obtained using a similar process; however, it is omitted from GGB Code 5 for simplicity. The complete algorithm is available in the supplementary material under the file *ConicalElbowAlg*.

To compute the conical elbow pattern, we followed a process similar to that used for the cylindrical elbow pattern. However, the conical elbow introduces slightly greater complexity, as demonstrated in the following code.

```

1 # Whole cone plus section planes
2 Vpat = Intersect(Gen, ejeCone)
3 End1Genbis = PAC - Distance(PortA1,PortA2)/2 vBase
4 Genbis = Line(Vpat, End1Genbis)
5 BasePat = Circle(PAC, Distance(End1Gen, PAC))
6 uLeft = UnitVector(Vector(End1Gen, Vpat))
7 Plane1PatLeft = End1Gen + uLeft Ferrule1Bottom
8 segmentsLeft = Sequence(If(Mod(i, 2)==0,
9   segmentsBottom(i), segmentsUpper(i)), i, 1,
10  nFerrules-2)
9 PlanesPatLeft = Sequence(Plane1PatLeft + uLeft Sum(
10   segmentsLeft, i), i, 1, nFerrules-2)
11 allPlanesPatLeft = Join({Plane1PatLeft}, PlanesPatLeft)
12 LinesPlanesPat = Sequence( Line(allPlanesPatLeft(i),
13   allPlanesPatRight(i)), i, 1, nFerrules-1)
14 # Flattened whole cone
15 GenLength = Distance(End1Gen, Vpat)
16 LBase = pi Distance(PortA1,PortA2)
17 angConeFlat = LBase/GenLength
18 End1GenbisRot = Rotate(End1Genbis, angConeFlat, Vpat)
19 BaseFlat = CircularArc(Vpat, End1Genbis, End1GenbisRot)
20 # Flattened section curves
21 PBaseRef = Point(BasePat)
22 arcBaseRef = Arc(BasePat, End1Genbis, PBaseRef)
23 GenPBaseRef = Ray(Vpat, PBaseRef)
24 PBaseRefFlat = Rotate(End1Genbis, Length(arcBaseRef)/
25   GenLength, Vpat)

```

```

23 GenPBaseRefFlat = Ray(Vpat, PBaseRefFlat)
24 PlanesWGenPBaseRef = Sequence(Intersect(GenPBaseRef,
25   LinesPlanesPat(i)), i, 1, nFerrules-1)
26 DPlanesWGenPBaseRef = Sequence(Distance(Intersect(Line(
27   PlanesWGenPBaseRef(i), PatRef), Genbis), Vpat),
28   i, 1, nFerrules-1)
29 PlanesLocGen = Sequence(Vpat + UnitVector(Vector(Vpat,
30   PBaseRefFlat)) DPlanesWGenPBaseRef(i), i, 1,
31   nFerrules-1)
32 PlanesFlat = Sequence(Locus(PlanesLocGen(i), PBaseRef),
33   i, 1, nFerrules-1)

```

GGB code 6 pattern of conical elbow (second paradigm)

The first block (lines 1–16) completes the computation of the whole cone and its section planes, which are used to derive the flattened state of all ferrules. The cone's right generatrix is defined using the vertex, *Vpat*, and the right endpoint of the base diameter, *End1Genbis*. After defining the mouth, *BasePat*, with diameter *D1* centered at *PAC*, the lengths of the ferrule generatrices to be mapped onto the cone's left generatrix are determined. These define the left endpoints of the section planes, *AllPlanesPatLeft*. A similar process is applied to the right generatrix (not shown in GGB code 6 but included in *ConicalElbowAlg* to compute *AllPlanesPatRight*. Finally, the algorithm calculates the list of lines containing the section planes, *LinesPlanesPat*.

The flat pattern of the revolution cone is a circular sector bounded by the arcs *TopFlat* and *BaseFlat*, and the generatrices *GenLeftFlat* and *GenRightFlat* as shown in Fig. B.15 (right), which summarizes the main objects produced by the algorithm. Its computation involves determining the angle, *angConeFlat*, which is used to rotate the point *End1Genbis* on *GenLeftFlat* to *End1GenbisRot* on *GenRightFlat*. This rotation allows for the computation of the *BaseFlat* circular arc (line 17).

The flattened section curves are calculated using Eq. (A.1), where $\Omega \equiv \mathcal{A}$ corresponds to *BasePat* (line 5), and ω corresponds to *PBaseRef* (line 19). After transporting the generatrix defined by *PBaseRef*, *GenPBaseRef*, to the flattened state, (*GenPBaseRefFlat*), the list of points where the section planes intersect the 3D generatrix (*PlanesWGenPBaseRef*) is also transported to obtain *PlanesLocGen* (Fig. B.15 - right), referred to as P_r in Eq. (A.1). The flattened curves $L(\omega)$ are computed as *PlanesFlat* (line 27).

Appendix C. Supplementary data

Supplementary material related to this article can be found online at <https://doi.org/10.1016/j.gmod.2024.101253>.

Data availability

No data was used for the research described in the article.

References

- [1] R. Migliari, Descriptive geometry: From its past to its future, *Nexus Netw. J.* 14 (3) (2012) 555–571.
- [2] P. Surynková, Central and parallel projections of regular surfaces: Geometric constructions using 3D modeling software, 2013, pp. 169–176.
- [3] E. Gutiérrez de Ravé, F.J. Jiménez-Hornero, A 3D descriptive geometry problem-solving methodology using CAD and orthographic projection, *Symmetry* 16 (4) (2024) 476.
- [4] A. Martín-Pastor, R. Vargas-Peña, Geometría descriptiva y pensamiento gráfico aumentado, *EGE-Expresión Gráfica en la Edificación* (20) (2024) 4–21, <http://dx.doi.org/10.4995/ega.2024.20720>.
- [5] F. González-Quintial, A. Martín-Pastor, Superficies rectificantes. Concepto, realidad geométrica y distorsión constructiva, *EGA Revista de Expresión Gráfica Arquitectónica* 28 (47) (2023) 228–239, <http://dx.doi.org/10.4995/ega.2023.16997>.
- [6] U. Kortenkamp, Foundations of Dynamic Geometry (Ph.D. thesis), Swiss Federal Institute of Technology Zurich, 1999.
- [7] C.H. Chivai, A.A. Soares, P. Catarino, Application of GeoGebra in the teaching of descriptive geometry: Sections of solids, *Mathematics* 10 (17) (2022) <http://dx.doi.org/10.3390/math10173034>.
- [8] K.A. Wottreng, Computer Methods in Descriptive and Differential Geometry Monge's Legacy (Ph.D. thesis), 1999.
- [9] I.E. Sutherland, Sketchpad, a Man-Machine Graphical Communication System (Ph.D. thesis), 1963.
- [10] M. Prado-Velasco, R. Ortiz-Marín, L. García-Ruesgas, M.G.D. Río-Cidoncha, Graphical modelling with computer extended descriptive geometry (CeDG): Description and comparison with CAD, *Comput.-Aided Des. Appl.* 18 (2) (2021) 272–284, <http://dx.doi.org/10.14733/cadaps.2021.272-284>.
- [11] K. Suzuki, Solving applied geometric design problems with the use of 3D-CAD, in: *Advances in Intelligent Systems and Computing*, Springer, 2018.
- [12] Q. Klogjeri, P. Klogjeri, Geogebra: A vital bridge linking mathematics with biology and other sciences, *SM J. Biol.* 3 (1) (2017) 272–284.
- [13] M. Hohenwarter, K. Fuchs, Combination of dynamic geometry, algebra and calculus in the software system GeoGebra, in: *Computer Algebra Systems and Dynamic Geometry Systems in Mathematics Teaching*, Sarvari, Cs. Hrsg., Pecs, Hungary, 2005, pp. 128–133.
- [14] F. Izquierdo Asensi, *Geometría Descriptiva Superior y Aplicada [Superior and Applied Descriptive Geometry]*, cuarta, , Ed. Paraninfo, 1996, p. 642.
- [15] C. Cobos Gutiérrez, A. Rodríguez Domínguez, J. Martín Salinas, *Geometría para Ingenieros. Tomo I Representación Diédrica*, Tébar, Madrid, 2001.
- [16] F.M. Croft, The need (?) for descriptive geometry in a world of 3D modeling, *Eng. Des. Graph. J.* 62 (3) (1998) 4–8.
- [17] M.V. Voronina, Z.O. Tretyakova, O.N.M. and, A.I. Folomkina, Descriptive geometry in educational process of technical university in Russia today, *Int. J. Environ. Sci. Educ.* 11 (18) (2016) 10911–10922.
- [18] H. Stachel, The status of today's descriptive geometry related education (CAD/CG/DG) in Europe, *J. Graph. Sci. Japan* 41 (2007) 15–20, <http://dx.doi.org/10.5989/jsgs.41.Supplement1.15>.
- [19] R. Ortiz-Marín, G. Del Río-Cidoncha, J. Martínez-Palacios, Where is descriptive geometry heading? in: *Proceedings of the XXIX International Congress INGE-GRAF*, in: *Advances in Design Engineering*, Springer International Publishing, pp. 365–373.
- [20] M. Prado-Velasco, R. Ortiz-Marín, Comparison of computer extended descriptive geometry (CeDG) with CAD in the modeling of sheet metal patterns, *Symmetry* 13 (4) (2021) 685, URL <https://www.mdpi.com/2073-8994/13/4/685>.
- [21] M. Prado-Velasco, L. García-Ruesgas, Intersection and flattening of surfaces in 3D models through computer-extended descriptive geometry (CeDG), *Symmetry* 15 (5) (2023) 984, URL <https://www.mdpi.com/2073-8994/15/5/984>.
- [22] M.A. Abánades, F. Botana, A. Montes, T. Recio, An algebraic taxonomy for locus computation in dynamic geometry, *Comput. Aided Des.* 56 (2014) 22–33, <http://dx.doi.org/10.1016/j.cad.2014.06.008>.
- [23] J.I. Rojas-Sola, G. del Río-Cidoncha, R. Ortiz-Marín, J.M. López-Pedregal, Design and development of sheet-metal elbows using programming with visual basic for applications in CATIA, *Symmetry* 13 (1) (2020) <http://dx.doi.org/10.3390/sym13010033>.
- [24] M. Prado-Velasco, L. García-Ruesgas, R. Ortiz-Marín, E. Vázquez-López, *Modelado paramétrico computacional de sistemas 3D mediante Geometría Descriptiva (CeDG): sistema diédrico*, Editorial Aula Magna, McGraw-Hill Interamericana de España, 2023, p. 328.
- [25] M. Prado-Velasco, L. García-Ruesgas, Extension of the dynamic geometry software for CeDG support and application to the accurate flattening of developable spatial surfaces (in press), in: *Advances in Design Engineering IV*, Springer International Publishing, 2025.
- [26] M. Prado-Velasco, L. García-Ruesgas, Discrete variable conditional 3D models in computer extended descriptive geometry (CeDG): Preliminary study on cylindrical polygonal sheet-metal elbow, in: C. Manchado del Val, M. Suffo Pino, R. Miralbes Buil, D. Moreno Sánchez, D. Moreno Nieto (Eds.), *Advances in Design Engineering IV*, Springer Nature Switzerland, 2024, pp. 170–179, http://dx.doi.org/10.1007/978-3-031-51623-8_17.
- [27] B. Leighton Wellman, *Geometría Descriptiva [Technical Descriptive Geometry]*, Editorial Reverté, S.A., 1987, p. 615.